MASTER OF COMPUTER APPLICATIONS — SEMESTER II

Academic Year 2025 – 2026

Unified Knowledge Library

Phase 2

Table Structure & Data Dictionary

Document	Table Structure & Data Dictionary
Database	`ukl_phase2	PostgreSQL 14+`
Extensions	`pgvector (AI vector embeddings)`

Revision History

Version	Date	Author(s)	Summary of Changes
Phase 1	Aug 2025	Anuja Patil	Initial schema: users, ai_models, chats, messages, notes, note_folders. MySQL. Flat structure, no hierarchy.
Phase 2	Feb 2026	Anuja Patil	Migrated to PostgreSQL 14+. Added: folders (hierarchical, self-ref FK), user_api_keys (BYOK AES-256-GCM), user_sessions (JWT revocation), token_usage (daily limit enforcement). Added pgvector embeddings to notes and messages. Soft-delete (deleted_at) applied to all tables. Brute-force protection on users. user_role and message_role ENUMs.

1. Overview

This document provides the complete table structure and data dictionary for the Unified Knowledge Library (UKL) Phase 2 database. The database is implemented in PostgreSQL 14+ and extends the Phase 1 schema with five major additions: a hierarchical folder system using self-referencing foreign keys, an AI memory layer using the pgvector extension for semantic similarity search (Retrieval-Augmented Generation), an encrypted API key store for the Bring Your Own Key (BYOK) feature, a session table enabling proper JWT refresh-token revocation, and a token usage tracking table to enforce per-user daily token limits.

Phase 1 used MySQL with flat, independent tables for chats and notes. Phase 2 migrates to PostgreSQL and organises all data inside a folder hierarchy — every chat and every note belongs to a folder, and folders can be nested to unlimited depth.

Phase Change Summary

Table Name	Phase 2 Status	Summary of Change
users	UPDATED	Added: role ENUM, last_login_at, daily_token_limit (+CHECK), is_limit_enabled, failed_login_attempts, locked_until, deleted_at, token expiry columns. Partial index on active users.
ai_models	UPDATED	Added: description, max_context_tokens (+CHECK), updated_at. Provider CHECK constraint. UNIQUE (provider, model_key).
folders	**NEW ★**	Entirely new. Hierarchical nesting via self-referencing parent_folder_id. Replaces flat note_folders from Phase 1.
chats	UPDATED	Added: folder_id, default_model_id, last_message_at. deleted_at replaces is_deleted. updated_at is app-managed (no DB trigger) — set by app when a new message is inserted.
messages	UPDATED	Added: token_count (+CHECK ≥0), status CHECK, embedding VECTOR(1536), embedding_status for async RAG pipeline.
notes	UPDATED	note_folder_id replaced by folder_id (FK → folders). Added embedding VECTOR(1536) and embedding_status for async RAG pipeline.
user_api_keys	**NEW ★**	Entirely new. AES-256-GCM encrypted API keys per user per provider. Includes key_label, is_revoked, expires_at, last_used_at. Provider CHECK consistent with ai_models.
user_sessions	**NEW ★**	Entirely new. Stores SHA-256 hashed refresh tokens for JWT revocation. Phase 1 stateless JWTs could not be revoked before expiry.
token_usage	**NEW ★**	Entirely new. One row per AI request. Tracks tokens_used per user per day to enforce daily_token_limit. Also links to chat_id and message_id for per-chat usage history in the UI.
~~note_folders~~	**REMOVED**	Deprecated and removed. Replaced by the folders table which supports unlimited hierarchy.

2. Key Legend

Symbol	Meaning
PK	Primary Key — uniquely identifies each row in the table. Auto-incremented using BIGSERIAL or SERIAL.
FK	Foreign Key — references the primary key of another table. Enforces referential integrity.
UK	Unique Key — the value must be unique across all rows in the column.

Symbol	Meaning
—	No key constraint on this column.

3. Custom ENUM Types

PostgreSQL does not support inline ENUM definitions like MySQL. These types must be created before running any table DDL.

Type Name	Allowed Values	Used In	Notes
`message_role`	`'user' | 'assistant' | 'system'`	`messages.role`	Required for LangChain context formatting across all AI providers. Create before the messages table.
`user_role`	`'user' | 'admin'`	`users.role`	★ **NEW Phase 2.** ENUM prevents typos like 'Admin'. To add values later: ALTER TYPE user_role ADD VALUE 'moderator';

4. Entity Relationships

Every foreign key relationship in the Phase 2 schema is listed below. The On Delete column defines what happens to child rows when the parent row is deleted.

Parent Table	Child Table	Cardinality	On Delete	Explanation
users	folders	One → Many	**CASCADE**	Deleting a user deletes all their folders, and by cascade all chats and notes inside them.
users	chats	One → Many	**CASCADE**	Deleting a user deletes all their chat sessions and messages.
users	messages	One → Many	SET NULL	User messages are retained; user_id set to NULL.
users	notes	One → Many	**CASCADE**	Deleting a user deletes all their notes.
users	user_api_keys	One → Many	**CASCADE**	Deleting a user permanently removes all stored encrypted API keys.
users	user_sessions	One → Many	**CASCADE**	Deleting a user invalidates all active login sessions.

Parent Table	Child Table	Cardinality	On Delete	Explanation
users	token_usage	One → Many	CASCADE	★ **NEW.** Deleting a user removes all token usage records.
folders	folders	One → Many (self)	CASCADE	Deleting a folder recursively deletes all sub-folders and their contents.
folders	chats	One → Many	SET NULL	If a folder is deleted, chats are kept but folder_id is set to NULL.
folders	notes	One → Many	SET NULL	If a folder is deleted, notes are kept but folder_id is set to NULL.
chats	messages	One → Many	CASCADE	Deleting a chat removes all messages in that conversation.
chats	notes	One → Many	SET NULL	Notes linked to a deleted chat are kept; chat_id set to NULL.
chats	token_usage	One → Many	SET NULL	★ **NEW.** Token records are kept when a chat is deleted; chat_id set to NULL.
ai_models	messages	One → Many	SET NULL	Removing a model does not delete chat history.
ai_models	chats	One → Many	SET NULL	default_model_id set to NULL if model is removed.
ai_models	token_usage	One → Many	SET NULL	★ **NEW.** Token records are kept when a model is removed; model_id set to NULL.
messages	notes	One → Many	SET NULL	Notes linked to a deleted message are kept; message_id set to NULL.
messages	token_usage	One → Many	SET NULL	★ **NEW.** Token records are kept when a message is deleted; message_id set to NULL.

5. Table Definitions

★ NEW marks columns added in Phase 2. ★ UPD marks columns modified from Phase 1.

5.1 users — UPDATED IN PHASE 2

Root table for all user accounts and bcrypt-hashed credentials. Phase 2 adds role, token limits, brute-force protection, soft delete, and token expiry columns.

Indexes: `idx_users_email (email)`, `idx_users_role (role)`, `idx_users_active (id) WHERE deleted_at IS NULL`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique user identifier. Primary key for all FK references from other tables.
name	VARCHAR(100)	No	—	–	Full display name shown in the interface.
email	VARCHAR(255)	No	**UK**	–	Unique email address used for login and account identification.
password_hash	VARCHAR(255)	No	—	–	Bcrypt-hashed password. The original password is never stored.
is_verified	BOOLEAN	No	—	FALSE	TRUE after the user completes email verification. Unverified accounts have restricted access.
role	user_role	No	—	'user'	★ **NEW** ENUM: 'user' | 'admin'. Controls platform access level. ENUM prevents invalid values like 'Admin'.
email_verify_token	VARCHAR(255)	Yes	—	NULL	One-time token sent to the user email for verification. Cleared after successful verification.
email_verify_token_expires_at	TIMESTAMP	Yes	—	NULL	★ **NEW** Expiry for the email verification link. Tokens older than 24 hours are rejected.
reset_password_token	VARCHAR(255)	Yes	—	NULL	Token issued during the password reset flow. Cleared after password is updated.
reset_password_token_expires_at	TIMESTAMP	Yes	—	NULL	★ **NEW** Expiry for the password reset link.

Column Name	Data Type	Nullable	Key	Default	Description
					Tokens older than 1 hour are rejected.
last_login_at	TIMESTAMP	Yes	—	NULL	★ **NEW** Timestamp of the most recent successful login. Used for security audit and session dashboards.
daily_token_limit	INTEGER	No	—	10000	★ **NEW** Maximum AI tokens the user may consume per day. Enforced when is_limit_enabled is TRUE. CHECK > 0.
is_limit_enabled	BOOLEAN	No	—	TRUE	★ **NEW** When TRUE, daily_token_limit is enforced via token_usage table. Admin accounts may set FALSE for unlimited access.
failed_login_attempts	INTEGER	No	—	0	★ **NEW** Counter incremented on each failed login. Reset to 0 on successful login. CHECK >= 0. Used with locked_until for brute-force protection.
locked_until	TIMESTAMP	Yes	—	NULL	★ **NEW** NULL = account not locked. Set to a future timestamp when failed_login_attempts exceeds threshold. Login is rejected until NOW() > locked_until.
deleted_at	TIMESTAMP	Yes	—	NULL	★ **NEW** Soft-delete timestamp. NULL = active account. Set to current timestamp on account deletion. Enables GDPR-compliant data handling.

Column Name	Data Type	Nullable	Key	Default	Description
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the account was first registered.
updated_at	TIMESTAMP	Yes	—	NOW()	Last modification timestamp. Maintained automatically by trigger_set_timestamp().

5.2 ai_models — UPDATED IN PHASE 2

Registry of AI model configurations. Phase 2 adds description, max_context_tokens, updated_at, provider CHECK, and UNIQUE (provider, model_key).

Indexes: `idx_ai_models_active (is_active)`, `idx_ai_models_provider (provider)`, `UNIQUE uq_ai_model (provider, model_key)`

Column Name	Data Type	Nullable	Key	Default	Description
id	SERIAL	No	**PK**	auto-inc	Unique model identifier.
name	VARCHAR(100)	No	—	–	Human-readable label shown to users (e.g., GPT-4o, Gemini 1.5 Pro, Llama 3 70B).
provider	VARCHAR(50)	No	—	–	★ **UPD** Vendor name for LangChain routing. CHECK: 'openai' | 'gemini' | 'groq' | 'anthropic'. Must match user_api_keys.provider.
model_key	VARCHAR(100)	No	—	–	★ **UPD** API identifier passed to the provider (e.g., gpt-4o, gemini-1.5-pro). Combined with provider in UNIQUE constraint.
description	TEXT	Yes	—	NULL	★ **NEW** Optional description shown in the model picker UI.
max_context_tokens	INTEGER	Yes	—	NULL	★ **NEW** Maximum context window in tokens. Used to truncate conversation history before API calls. CHECK > 0.
is_active	BOOLEAN	No	—	TRUE	When FALSE, the model is hidden from users without removing its configuration.

Column Name	Data Type	Nullable	Key	Default	Description
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when this model configuration was added.
updated_at	TIMESTAMP	Yes	—	NOW()	★ **NEW** Last modification timestamp. Maintained by trigger_set_timestamp().

5.3 folders — NEW IN PHASE 2

Hierarchical workspace containers. Unlimited nesting via self-referencing parent_folder_id. Replaces the flat note_folders table from Phase 1.

Indexes: `idx_folders_active (user_id, deleted_at)`, `idx_folders_parent (parent_folder_id)`, `idx_folders_sidebar (user_id, deleted_at, pinned DESC, updated_at DESC)`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique folder identifier.
user_id	BIGINT	No	**FK**	–	References users(id). Every folder is owned by exactly one user. CASCADE on user delete.
parent_folder_id	BIGINT	Yes	**FK**	NULL	References folders(id) — self-referencing. NULL = top-level folder. Non-NULL = nested inside another folder. CASCADE on parent delete removes all child folders recursively.
name	VARCHAR(255)	No	—	–	User-defined display name shown in the sidebar.
color	VARCHAR(20)	Yes	—	'#5227FF'	Hex colour code for the folder icon. Defaults to UKL brand purple.
pinned	BOOLEAN	Yes	—	FALSE	TRUE pins the folder to the top of the sidebar.
deleted_at	TIMESTAMP	Yes	—	NULL	Soft-delete timestamp. NULL = active. Enables audit trail and data recovery.
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the folder was created.
updated_at	TIMESTAMP	Yes	—	NOW()	Last modification timestamp. Maintained by trigger_set_timestamp().

Recursive Folder Query — Fetch Complete Folder Tree in One Call

```
WITH RECURSIVE folder_tree AS (
  SELECT * FROM folders
  WHERE user_id = $1 AND parent_folder_id IS NULL
  UNION ALL
  SELECT f.* FROM folders f
  JOIN folder_tree ft ON f.parent_folder_id = ft.id
)
SELECT * FROM folder_tree WHERE deleted_at IS NULL;
```

5.4 chats — UPDATED IN PHASE 2

Individual AI conversation sessions. Phase 2 adds folder_id, default_model_id, and last_message_at. deleted_at replaces the Phase 1 is_deleted boolean. updated_at is app-managed — the DB trigger is intentionally absent because the common update path is inserting a new message (a separate INSERT), not updating the chats row directly.

Indexes: `idx_chats_user (user_id)`, `idx_chats_user_active (user_id, deleted_at)`, `idx_chats_sidebar (user_id, deleted_at, pinned DESC, last_message_at DESC)`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique chat session identifier.
user_id	BIGINT	No	**FK**	–	References users(id). Owner of this chat session. CASCADE on user delete.
folder_id	BIGINT	Yes	**FK**	NULL	★ **NEW** References folders(id). The workspace folder this chat belongs to. SET NULL when folder is deleted.
default_model_id	INTEGER	Yes	**FK**	NULL	★ **NEW** References ai_models(id). Default AI model for this conversation. SET NULL if model is removed.
title	VARCHAR(255)	No	—	–	Display title shown in the sidebar. Set by user or auto-generated by AI.
pinned	BOOLEAN	Yes	—	FALSE	TRUE pins the chat to the top of the sidebar.
last_message_at	TIMESTAMP	Yes	—	NULL	★ **NEW** Timestamp of the most recent message. Set by

Column Name	Data Type	Nullable	Key	Default	Description
					the app layer when a new message is inserted. Used for sidebar sort order.
is_auto_title_generated	BOOLEAN	Yes	—	FALSE	TRUE when the title was generated by the AI rather than manually entered by the user.
deleted_at	TIMESTAMP	Yes	—	NULL	Soft-delete timestamp. NULL = active. Query with deleted_at IS NULL to fetch active chats.
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the chat session was first created.
updated_at	TIMESTAMP	Yes	—	NOW()	★ **UPD** App-managed timestamp. No DB trigger — the app sets this when adding a new message to the chat.

5.5 messages — UPDATED IN PHASE 2

Individual messages within a chat. Role is either 'user' or 'assistant'. Phase 2 adds token_count, status, embedding, and embedding_status for async RAG.

Indexes: `idx_messages_chat (chat_id, deleted_at, created_at)`, `idx_messages_pinned`, `idx_messages_status WHERE status != completed`, `idx_messages_embedding (IVFFlat cosine, lists=100)`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique message identifier.
chat_id	BIGINT	No	**FK**	–	References chats(id). The conversation this message belongs to. CASCADE on chat delete.
user_id	BIGINT	Yes	**FK**	NULL	References users(id). Populated for user messages. NULL for all AI-generated messages.
model_id	INTEGER	Yes	**FK**	NULL	References ai_models(id). Records which model generated this response. NULL for user messages.

Column Name	Data Type	Nullable	Key	Default	Description
role	`message_role`	No	—	–	ENUM: 'user' | 'assistant' | 'system'. Required for LangChain context formatting.
content	`TEXT`	No	—	–	Full body text of the message. No length limit.
token_count	`INTEGER`	Yes	—	`NULL`	★ **NEW** Tokens used by the AI response. Populated for assistant messages. CHECK >= 0. Stored in token_usage table for daily limit enforcement.
status	`VARCHAR(20)`	No	—	`'completed'`	★ **NEW** CHECK: 'pending' | 'completed' | 'failed'. Tracks generation state. 'pending' while streaming; 'failed' on API error.
embedding	`VECTOR(1536)`	Yes	—	`NULL`	★ **NEW** OpenAI text-embedding-3-small vector (1536 dims). Populated asynchronously. Enables cosine similarity search across message history.
embedding_status	`VARCHAR(10)`	No	—	`'pending'`	★ **NEW** CHECK: 'pending' | 'done' | 'failed'. Tracks async embedding pipeline state. 'done' = ready for <=> search.
pinned	`BOOLEAN`	No	—	`FALSE`	TRUE marks the message as pinned for quick reference within the chat.
deleted_at	`TIMESTAMP`	Yes	—	`NULL`	Soft-delete timestamp. Preserves chat history for recovery.
created_at	`TIMESTAMP`	Yes	—	`NOW()`	Timestamp when the message was sent or generated. No updated_at — messages are immutable once sent.

5.6 notes — UPDATED IN PHASE 2

UKL Phase 2 — Table Structure & Data Dictionary

User-created knowledge notes linked to a folder, chat, or individual message. Phase 2 replaces note_folder_id with folder_id, and adds embedding + embedding_status for the async RAG pipeline.

Indexes: `idx_notes_user`, `idx_notes_folder`, `idx_notes_chat`, `idx_notes_pinned`, `idx_notes_embedding_status WHERE deleted_at IS NULL`, `idx_notes_embedding (IVFFlat cosine, lists=100)`

Column Name	Data Type	Nullable	Key	Default	Description
id	`BIGSERIAL`	No	**PK**	`auto-inc`	Unique note identifier.
user_id	`BIGINT`	No	**FK**	—	References users(id). Owner of the note. CASCADE on user delete.
folder_id	`BIGINT`	Yes	**FK**	`NULL`	★ **UPD** References folders(id). Replaces note_folder_id from Phase 1. Links the note to a hierarchical folder. SET NULL when folder is deleted.
chat_id	`BIGINT`	Yes	**FK**	`NULL`	References chats(id). Optional — links the note to an entire conversation. SET NULL when chat is deleted.
message_id	`BIGINT`	Yes	**FK**	`NULL`	References messages(id). Optional — links the note to a specific AI response. SET NULL when message is deleted.
title	`VARCHAR(255)`	Yes	—	`NULL`	Optional heading for the note. Can be left blank.
content	`TEXT`	No	—	—	Full body text. This is the content that is embedded and indexed for RAG retrieval.
embedding	`VECTOR(1536)`	Yes	—	`NULL`	★ **NEW** OpenAI text-embedding-3-small vector (1536 dims). Populated asynchronously via embedding job.
embedding_status	`VARCHAR(10)`	No	—	`'pending'`	★ **NEW** CHECK: 'pending' | 'done' | 'failed'. 'pending' = saved but not yet embedded; 'done' = ready for <=> search; 'failed' = retry required.
color	`VARCHAR(20)`	Yes	—	`'#5227FF'`	Hex colour code for visual distinction in the notes list UI.

Column Name	Data Type	Nullable	Key	Default	Description
pinned	BOOLEAN	Yes	—	FALSE	TRUE pins the note to the top of the notes panel.
deleted_at	TIMESTAMP	Yes	—	NULL	Soft-delete timestamp. NULL = active. Supports audit trail and GDPR compliance.
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the note was created.
updated_at	TIMESTAMP	Yes	—	NOW()	Last modification timestamp. Maintained by trigger_set_timestamp().

RAG Similarity Search — Top 5 Relevant Notes for a Query

```
SELECT id, title, content
FROM notes
WHERE folder_id = $1
  AND deleted_at IS NULL
  AND embedding IS NOT NULL
ORDER BY embedding <=> $2    -- $2 = query vector from text-embedding-3-small
LIMIT 5;

-- Retrieved content is prepended to the AI prompt as context before generation.
```

5.7 user_api_keys — NEW IN PHASE 2

AES-256-GCM encrypted API keys for the BYOK feature. One record per user per provider. The original API key is never stored — only the three outputs of the encryption process: iv, auth_tag, ciphertext.

Indexes: `idx_user_api_keys_user (user_id)`, `idx_api_keys_expiry (expires_at)`, `UNIQUE uq_user_provider (user_id, provider)`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique key record identifier.
user_id	BIGINT	No	**FK**	–	References users(id). CASCADE on user delete — all keys removed when account is deleted.
provider	VARCHAR(50)	No	—	–	CHECK: 'openai' | 'gemini' | 'groq' | 'anthropic'. Matches ai_models.provider. Combined with

Column Name	Data Type	Nullable	Key	Default	Description
					user_id in UNIQUE constraint — one key per provider per user.
key_label	VARCHAR(100)	Yes	—	NULL	Optional user-defined label (e.g., 'Work Key', 'Personal'). Displayed in the Settings UI.
iv	VARCHAR(64)	No	—	–	AES-256-GCM Initialisation Vector stored as hex. Randomly generated fresh for every encryption. Required for decryption.
auth_tag	VARCHAR(64)	No	—	–	GCM Authentication Tag as hex. Verifies the ciphertext has not been tampered with. Decryption is rejected if this tag does not match.
ciphertext	TEXT	No	—	–	The encrypted API key as hex. Cannot be decrypted without the MASTER_ENCRYPTION_KEY server environment variable.
is_revoked	BOOLEAN	No	—	FALSE	TRUE marks the key as revoked without deleting it. Revoked keys are ignored by the API routing layer.
expires_at	TIMESTAMP	Yes	—	NULL	Optional expiry. NULL = no expiry. Keys past this timestamp are treated as revoked.
last_used_at	TIMESTAMP	Yes	—	NULL	Timestamp of the most recent successful API call using this key. Used for activity monitoring.
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the key was first saved.
updated_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the key was last updated. Tracks key rotation history.

BYOK Key Rotation — Upsert Pattern

```
INSERT INTO user_api_keys (user_id, provider, iv, auth_tag, ciphertext)
VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (user_id, provider) DO UPDATE
  SET iv         = EXCLUDED.iv,
      auth_tag   = EXCLUDED.auth_tag,
      ciphertext = EXCLUDED.ciphertext,
```

```
        is_revoked = FALSE,
        updated_at = NOW();
```

5.8 user_sessions — NEW IN PHASE 2

SHA-256 hashed refresh tokens for JWT revocation. Phase 1 stateless JWTs could not be revoked before expiry. This table makes logout truly effective.

Indexes: `idx_sessions_user (user_id, is_revoked)`, `idx_sessions_token (token_hash)`, `idx_sessions_expiry (expires_at)`, `idx_sessions_active (user_id, expires_at) WHERE is_revoked = FALSE`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique session record identifier.
user_id	BIGINT	No	**FK**	—	References users(id). CASCADE on user delete — all sessions invalidated when account is deleted.
token_hash	VARCHAR(255)	No	**UK**	—	SHA-256 hash of the raw refresh token. The raw token is NEVER stored — only its hash. Looked up on every /refresh request.
ip_address	VARCHAR(45)	Yes	—	NULL	Client IPv4 or IPv6 address at login. VARCHAR(45) correctly supports full IPv6 addresses. Used for security audit.
user_agent	TEXT	Yes	—	NULL	Browser and device info from the User-Agent header at login. Displayed in the active-sessions security dashboard.
is_revoked	BOOLEAN	No	—	FALSE	TRUE = logged out or manually revoked. The /refresh endpoint rejects revoked sessions immediately.
expires_at	TIMESTAMP	No	—	NOW()+30d	Hard expiry set to NOW() + 30 days on issue. Sessions past this timestamp are rejected even if is_revoked is FALSE.
last_used_at	TIMESTAMP	Yes	—	NULL	Timestamp of the most recent successful token refresh. Updated on every /refresh call.
created_at	TIMESTAMP	Yes	—	NOW()	Timestamp when the session was created (login time). No updated_at

Column Name	Data Type	Nullable	Key	Default	Description
					— sessions are append-only; only is_revoked changes.

5.9 token_usage — NEW IN PHASE 2

Tracks per-user daily AI token consumption to enforce users.daily_token_limit. One row per AI request. Also links to chat_id and message_id enabling per-chat usage history in the UI.

Indexes: `idx_token_usage_user_date (user_id, used_date)`, `idx_token_usage_chat (chat_id) WHERE chat_id IS NOT NULL`

Column Name	Data Type	Nullable	Key	Default	Description
id	BIGSERIAL	No	**PK**	auto-inc	Unique usage record identifier.
user_id	BIGINT	No	**FK**	—	References users(id). CASCADE on user delete — all usage records removed.
model_id	INTEGER	Yes	**FK**	NULL	References ai_models(id). Records which model generated the tokens. SET NULL if model is removed.
chat_id	BIGINT	Yes	**FK**	NULL	References chats(id). Links usage to a conversation. SET NULL if chat is deleted. Powers per-chat usage history view.
message_id	BIGINT	Yes	**FK**	NULL	References messages(id). Links usage to a specific AI response. SET NULL if message is deleted. Powers per-message token detail view.
tokens_used	INTEGER	No	—	—	Number of tokens consumed by this AI request. CHECK > 0.
used_date	DATE	No	—	CURRENT_DATE	Date the tokens were consumed. Grouped by day for daily limit enforcement. Stored as DATE (not TIMESTAMP) for efficient daily aggregation.
created_at	TIMESTAMP	Yes	—	NOW()	Exact timestamp of the AI request.

Token Usage Query Reference

```
-- Q1: Daily total for limit enforcement (run BEFORE each AI request)
SELECT COALESCE(SUM(tokens_used), 0) AS total
FROM token_usage
WHERE user_id = $1 AND used_date = CURRENT_DATE;

-- Q2: Per-conversation usage breakdown (UI dashboard)
SELECT chat_id, SUM(tokens_used) AS total_tokens, COUNT(*) AS requests
FROM token_usage
WHERE user_id = $1
GROUP BY chat_id ORDER BY total_tokens DESC;

-- Q3: Per-message token detail inside a chat
SELECT message_id, model_id, tokens_used, created_at
FROM token_usage
WHERE chat_id = $1
ORDER BY created_at;

-- Cleanup job (schedule nightly): remove old revoked sessions
DELETE FROM user_sessions WHERE is_revoked = TRUE OR expires_at < NOW();
```

6. Sample Data

Illustrative example rows. All values are representative only.

users

id	name	email	is_verified	role	daily_token_limit	failed_login_attempts	locked_until	deleted_at
1	Anuja Patil	anuja@ukl.app	TRUE	admin	50000	0	NULL	NULL
2	Akshata Shrivastava	akshata@ukl.app	TRUE	user	10000	0	NULL	NULL
3	Test User	test@ukl.app	FALSE	user	10000	5	2026-02-01 10:30:00	NULL

ai_models

id	name	provider	model_key	max_context_tokens	is_active	description
1	GPT-4o	openai	gpt-4o	128000	TRUE	OpenAI flagship multimodal model.
2	Gemini 1.5 Pro	gemini	gemini-1.5-pro	1000000	TRUE	Google long-context model.

id	name	provider	model_key	max_context_tokens	is_active	description
3	Llama 3 70B	groq	llama3-70b-8192	8192	TRUE	Open-source model via Groq.

token_usage

id	user_id	model_id	chat_id	message_id	tokens_used	used_date	created_at
1	1	1	3	12	850	2026-02-10	2026-02-10 14:22:05
2	1	1	3	14	1200	2026-02-10	2026-02-10 14:35:12
3	2	2	7	NULL	600	2026-02-10	2026-02-10 16:01:44

user_sessions

id	user_id	token_hash (SHA-256)	ip_address	is_revoked	expires_at	user_agent	...
1	1	a3f9d2...c841	192.168.1.10	FALSE	2026-03-12 09:00:00	Chrome 122 / Windows 11	
2	2	bb04e1...7f23	10.0.0.5	TRUE	2026-03-01 08:00:00	Safari 17 / macOS	

7. Soft Delete Pattern

All tables in Phase 2 use the `deleted_at TIMESTAMP NULL` pattern instead of the `is_deleted BOOLEAN` column from Phase 1.

Property	Details
Column type	`TIMESTAMP NULL DEFAULT NULL`
Active record	`deleted_at IS NULL`
Deleted record	deleted_at = timestamp of deletion
Why not BOOLEAN?	TIMESTAMP records when the deletion happened, enabling audit trails, data recovery windows, and GDPR-compliant scheduled purges. A boolean carries no time information.

Property	Details
Standard query	`WHERE deleted_at IS NULL` — append to every SELECT that should return only active records.
Applied to	users, folders, chats, messages, notes, user_api_keys (via is_revoked equivalent)

8. Auto-Update Trigger

A shared PostgreSQL function automatically sets `updated_at = NOW()` before every UPDATE, eliminating application-layer timestamp management.

trigger_set_timestamp() — Applied to: users, ai_models, folders, notes, user_api_keys (NOT chats — see note below)

```
CREATE OR REPLACE FUNCTION trigger_set_timestamp()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Example: applied to the notes table
CREATE TRIGGER set_timestamp_notes
  BEFORE UPDATE ON notes
  FOR EACH ROW EXECUTE FUNCTION trigger_set_timestamp();

-- WHY chats has NO trigger:
-- The common update path for a chat is inserting a new message
-- (INSERT on messages table), NOT an UPDATE on the chats row.
-- A trigger on chats would never fire for this case.
-- The app layer explicitly sets chats.updated_at = NOW() and
-- chats.last_message_at = NOW() when a new message is added.
```

— End of Table Structure & Data Dictionary —